Test Case ID	Title	Preconditions	Steps	Expected Result	Actual Result	Status	Bug ID/ Notes
TC-01	Regular and Command Completion with Single Dot	Java class with main method exists	1. Place caret inside method. 2. Write “System”class. 3. Type `.` 4. Observe suggestions.	Regular and command completion suggestions should appear.	Regular and command completion suggestions appear.	Pass	
TC-02	Command Completion with Double Dot	Java class with main method exists	1. Place caret inside method. 2. Write “System”class. 3. Type `..` 4. Observe suggestions.	Only command completion suggestions should appear.	Only command completion suggestions appear.	Pass	
TC-03	Command Completion Grouping	Java class with main method exists, Command grouping enabled in settings	1. Place caret inside method. 2. Write “System”class. 3. Type `.` 4. Check if command suggestions are grouped separately.	Command suggestions should be grouped separately.	Command suggestions are grouped separately.	Pass	
TC-04	Command Completion Search	Java class with main method exists	1. Place caret in code. 2. Write “System”class. 3. Type `.` 4. Start typing "get". 5. Observe filtered suggestions.	Suggestions should be filtered as you type.	Suggestions are filtered as you type.	Pass	
TC-05	Fixes for Issues (Quick-Fix)	Java class with main method exists	1. Introduce an error (e.g., use undefined variable). 2. Place caret on error. 3. Type `..`	Quick-fix suggestions should be shown.	Quick-fix suggestions are shown.	Pass	

			4. Check for quick-fix suggestions.				
TC-06a	Code Transformation (Extract Method)	Java class with a block of statements inside a method exists	1. Select a valid block of statements inside a method including some part of a statement or an incomplete expression. 2.Press Cmd+Option+M	A message should be displayed: "Unable to extract method. Selected block should represent a set of statements or an expression"	A message is shown: "Unable to extract method. Selected block should represent a set of statements or an expression"	Pass	
TC-06b	Code Transformation (Extract Method)	Java class with a block of statements inside a method exists	1. Select a valid block of statements inside a method. 2.Press Cmd+Option+M 3. Press Enter.	The selected statements should be extracted into a new method, and the original code should be replaced with a call to the new method.	The selected statements are extracted into a new method, and the original code is replaced with a call to the new method.	Pass	SUG-001
TC-06c	Code Transformation (Extract Method)	Java class with a block of statements inside a method exists	1. Select a valid block of statements inside a method. 2.Press Option+Enter 3. Select Extract -> Method.	The selected statements should be extracted into a new method, and the original code should be replaced with a call to the new method.	The selected statements are extracted into a new method, and the original code are replaced with a call to the new method.	Pass	
TC-07a	Undo "*Extract Method*" refactoring	Java class with main method exists, method has been extracted	1. Select a valid block of statements inside a method. 2.Press Cmd+Option+N	A message should be displayed: "Cannot perform refactoring.	A message is displayed: "Cannot perform refactoring.Caret should	Pass	

				Caret should be positioned at the name of element to be refactored".	be positioned at the name of element to be refactored".		
TC-07b	Undo "*Extract Method*" refactoring	Java class with main method exists, method has been extracted	1. Position caret at the name of new extraced method. 2.Press Cmd+Option+N 3. Select the relevant option from "Inlive Method" window 4. Click on "Refactor" button.	The code should be reverted to its original state before the Extract Method refactoring.	The code reverts to its original state before the Extract Method refactoring.	Pass	
TC-08	Refactoring and Navigation	Java class with main method exists	1. Place caret on variable/method name. 2. Type . . 3. Look for "Rename", "Move", etc action 4. Click on "Rename" 5. Write a new method name.	Refactoring/ navigation actions should be suggested, renaming should be done successfully.	Refactoring/ navigation actions are suggested, renaming is done successfully.	Pass	
TC-09	Code Generation (Getter/Setter)	Class with private fields exists	1. Place caret inside class. 2. Type . . 3. Select "Generate getters and setters".	Getters and setters should be generated.	Getters and setters are generated.	Pass	
TC-10	Code Generation (Constructor)	Class with fields exists	1. Place caret inside class. 2. Type . . 3. Select "Generate constructor".	Constructor should be generated.	Constructor is generated.	Pass	
TC-11	Code Generation (toString)	Class with fields exists	1. Place caret inside class. 2. Type . . 3. Select	toString should be generated.	toString is generated.	Pass	

			"Generate toString".				
TC-12	Command Completion in Read-Only Files (Go to declaration)	Read-only file open	1. Open read-only file (test example – External Libraries/java/util/ArrayList). 2. Place caret at end of any symbol. 3. Type . 4. Check for navigation/reference actions. 5. Click on “Go to declaration”.	Should navigate to declaration.	Is navigated to declaration.	Pass	
TC-13	Command Completion in Read-Only Files (Show usages)	Read-only file open	1. Open read-only file (test example – External Libraries/java/util/ArrayList). 2. Place caret at end of any symbol. 3. Type . 4. Check for navigation/reference actions. 5. Click on “Show usages”.	Should navigate to method usages.	Is navigated to method usages.	Pass	
TC-14	Command Completion in Read-Only Files (Go to super method)	Read-only file open	1. Open read-only file (test example – External Libraries/java/util/ArrayList). 2. Place caret at end of any symbol. 3. Type . 4. Check for navigation/reference actions. 5. Click on “Go to super method”.	Should navigate to super method.	Is navigated to super method.	Pass	
TC-15	High-Level Transformations	Java class with loop/if exists	1. Write a loop or if-statement. 2. Select block. 3. Look for	High-level transformations should	High-level transformations are suggested.	Pass	

			"Extract method", “Surround”.	be suggested.			
TC-16	Usability: Context Relevance	Java class exists	1. Place caret in various contexts. 2. Type . . 3. Check if suggestions are relevant to context.	Suggestions should be context-aware and relevant.	Suggestions are context-aware and relevant.	Pass	
TC-17	Usability: Preview of Changes	Java class exists	1. Place caret in code. 2. Type . . 3. Select a command. 4. Observe if a preview of the change is shown.	Preview of change should be displayed.	Preview of change is displayed.	Pass	
TC-18	Usability: Keyboard Navigation	Java class exists	1. Place caret in code. 2. Type . . 3. Use keyboard to navigate and select suggestions.	Keyboard navigation and selection should work smoothly.	Keyboard navigation and selection work smoothly.	Pass	

Suggestion ID: SUG-001
Title: Improve User Guidance During Extract Method Refactoring

Preconditions:
User selects a valid block of code and invokes "Extract Method" via Command Completion.

Steps to Reproduce:

1. Select a valid block of code inside a method.
2. Invoke Command Completion (. .) and choose "Extract method."
3. Observe that the selected code is highlighted in green and becomes temporarily read-only.

Observed Behavior:
The code remains highlighted in green and cannot be edited until the user presses Enter. There is no clear prompt or instruction indicating that pressing Enter will complete the refactoring. As a first-time user, it is not obvious that this is the required action, which can lead to confusion or the impression that the feature is not working.

Expected Behavior:
A clear, visible prompt or inline instruction should be provided, guiding the user to press Enter to complete the Extract Method refactoring. This would improve usability and reduce confusion for new users.

Status: Suggestion

Screenshot/Logs:

Suggestions for Deeper Testing

To ensure the robustness and versatility of the Command Completion feature, the following areas are recommended for further, in-depth testing:

- **Large and Complex Codebases:**
  Evaluate Command Completion in projects with a large number of classes, deeply nested packages, and complex dependencies. This will help assess the relevance and performance of suggestions in enterprise-scale environments.
- **Framework Integration:**
  Test Command Completion with popular Java frameworks such as Spring (e.g., bean wiring, annotations) and JUnit (e.g., test method generation, assertions). Verify that framework-specific suggestions are accurate and context-aware.
- **Performance with Large Files:**
  Open and edit very large Java files (thousands of lines) to measure the speed and responsiveness of Command Completion. Check for any lag, UI freezes, or memory issues.
- **Accessibility:**
  Assess compatibility with screen readers and other assistive technologies. Ensure all Command Completion features are accessible via keyboard navigation and that suggestion previews and popups are readable and navigable.
- **Kotlin Support:**
  Repeat exploratory testing in Kotlin files, as Command Completion is also implemented for Kotlin. Compare the feature set and quality of suggestions between Java and Kotlin.
- **Custom Code Styles and Settings:**
  Apply different code style configurations (such as formatting and naming conventions) and test if Command Completion respects and adapts to custom project or user settings.

Summary:
Deeper testing in these areas will help uncover edge cases, performance bottlenecks, and ensure that Command Completion is robust, accessible, and valuable across a wide range of real-world scenarios.